

AI ASSISTED CODING

ASSIGNMENT -16.4

Name: N.vaishnavi

Batch:44

H.No:2303A52405

Task 1: Database Schema Design for an
Online Course Platform

Prompt:

Generate SQL INSERT statements to populate a database with sample data. The data should include at least 3 instructors, 4 courses, and 3 students. Ensure the data is realistic and properly structured. Also include SELECT queries to verify that the data has been inserted correctly.

Code:

```
# assign164 tqsk1.sql
1 -- clean old tables
2 DROP TABLE IF EXISTS students;
3 DROP TABLE IF EXISTS courses;
4 DROP TABLE IF EXISTS instructors;
5
6 -- create tables
7 CREATE TABLE instructors (
8   id INTEGER PRIMARY KEY AUTOINCREMENT,
9   name TEXT,
10  position TEXT,
11  salary REAL
12 );
13
14 CREATE TABLE courses (
15   id INTEGER PRIMARY KEY AUTOINCREMENT,
16   name TEXT,
17   instructor_id INTEGER,
18   FOREIGN KEY (instructor_id) REFERENCES instructors(id)
19 );
20
21 CREATE TABLE students (
22   id INTEGER PRIMARY KEY AUTOINCREMENT,
23   name TEXT,
24   course_id INTEGER,
25   FOREIGN KEY (course_id) REFERENCES courses(id)
26 );
27
28 -- Insert instructors
29 INSERT INTO instructors(name, position, salary) VALUES
30 ('Alice Johnson', 'Instructor', 60000.00),
31 ('Bob Williams', 'Instructor', 62000.00),
32 ('Charlie Brown', 'Instructor', 58000.00);
33
34 -- Insert courses
35 INSERT INTO courses(name, instructor_id) VALUES
36 ('Introduction to Programming', 1),
37 ('Data Structures', 2),
38 ('Database Systems', 3),
39 ('Web Development', 1);
40
41 -- Insert students
42 INSERT INTO students(name, course_id) VALUES
43 ('David Smith', 1),
44 ('Emma Davis', 2),
45 ('Frank Miller', 3);
46
47 -- verify data
48 SELECT * FROM instructors;
49 SELECT * FROM courses;
50 SELECT * FROM students;
```

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

id	name	position	salary
1	Alice Johnson	Instructor	60000.0
2	Bob Williams	Instructor	62000.0
3	Charlie Brown	Instructor	58000.0

SQL v < 1 / 1 > 1 - 3 of 3

id	name	instructor_id
1	Introduction to Programming	1
2	Data Structures	2
3	Database Systems	3
4	Web Development	1

SQL v < 1 / 1 > 1 - 4 of 4

id	name	course_id
1	David Smith	1
2	Emma Davis	2
3	Frank Miller	3

SQL v < 1 / 1 > 1 - 3 of 3

Observation:

The SQL INSERT statements were successfully executed without errors in SQLite. Three instructors, four courses, and three students were added to their respective tables. The SELECT queries confirmed that all records were correctly inserted and displayed as expected. The relationships between courses and instructors were maintained using instructor_id.

Task 2: Populate the Database with Sample Records

Prompt:

Generate SQL INSERT statements to populate a database with sample records. The database should include at least 3 instructors, 4 courses, and 3 students. Ensure proper relationships between tables and include SELECT queries to verify that the data has been inserted correctly.

Code:

```
5: assign16.4 task2.sql
1  -- remove old tables (important)
2  DROP TABLE IF EXISTS students;
3  DROP TABLE IF EXISTS courses;
4  DROP TABLE IF EXISTS instructors;
5
6  -- create instructors table
7  CREATE TABLE instructors (
8      id INTEGER PRIMARY KEY AUTOINCREMENT,
9      name TEXT,
10     position TEXT,
11     salary REAL
12 );
13
14 -- create courses table (FOREIGN KEY used here)
15 CREATE TABLE courses (
16     id INTEGER PRIMARY KEY AUTOINCREMENT,
17     name TEXT,
18     instructor_id INTEGER,
19     FOREIGN KEY (instructor_id) REFERENCES instructors(id)
20 );
21
22 -- create students table (FOREIGN KEY used here)
23 CREATE TABLE students (
24     id INTEGER PRIMARY KEY AUTOINCREMENT,
25     name TEXT,
26     course_id INTEGER,
27     FOREIGN KEY (course_id) REFERENCES courses(id)
28 );
29
30 -- Insert instructors (3)
31 INSERT INTO instructors(name, position, salary) VALUES
32 ('Alice Johnson', 'Instructor', 60000.00),
33 ('Bob Williams', 'Instructor', 62000.00),
34 ('Charlie Brown', 'Instructor', 58000.00);
35
36 -- Insert courses (4)
37 INSERT INTO courses(name, instructor_id) VALUES
38 ('Introduction to Programming', 1),
39 ('Data Structures', 2),
40 ('Database Systems', 3),
41 ('Web Development', 1);
42
43 -- Insert students (3)
44 INSERT INTO students(name, course_id) VALUES
45 ('David Smith', 1),
46 ('Emma Davis', 2),
47 ('Frank Miller', 3);
48
49 -- verify data
50 SELECT * FROM instructors;
51 SELECT * FROM courses;
52 SELECT * FROM students;
```

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 3 of 3

id	name	position	salary
1	Alice Johnson	Instructor	60000.0
2	Bob Williams	Instructor	62000.0
3	Charlie Brown	Instructor	58000.0

SQL v < 1 / 1 > 1 - 4 of 4

id	name	instructor_id
1	Introduction to Programming	1
2	Data Structures	2
3	Database Systems	3
4	Web Development	1

SQL v < 1 / 1 > 1 - 3 of 3

id	name	course_id
1	David Smith	1
2	Emma Davis	2
3	Frank Miller	3

Observation:

The SQL INSERT statements were successfully executed to populate the database with sample records. Three instructors, four courses, and three students were added. The SELECT queries were used to verify the inserted data, and all records were displayed correctly without any errors. The relationships between instructors, courses, and students were maintained using foreign keys.

Task 3: Implement Core Database Operations (CRUD)

Prompt:

Generate SQL queries to perform CRUD operations on a database. The operations should include retrieving all courses, updating a student's email address, deleting a course, and verifying each operation using SELECT queries to ensure correctness.

Code:

```
# assign164 task3.sql
1 -- clean old tables
2 DROP TABLE IF EXISTS students;
3 DROP TABLE IF EXISTS courses;
4 DROP TABLE IF EXISTS instructors;
5
6 -- create tables
7 CREATE TABLE instructors (
8   id INTEGER PRIMARY KEY AUTOINCREMENT,
9   name TEXT
10 );
11
12 CREATE TABLE courses (
13   id INTEGER PRIMARY KEY AUTOINCREMENT,
14   name TEXT,
15   instructor_id INTEGER
16 );
17
18 CREATE TABLE students (
19   id INTEGER PRIMARY KEY AUTOINCREMENT,
20   name TEXT,
21   email TEXT,
22   course_id INTEGER
23 );
24
25 -- Insert data
26 INSERT INTO instructors(name) VALUES
27 ('Alice'), ('Bob'), ('Charlie');
28
29 INSERT INTO courses(name, instructor_id) VALUES
30 ('Programming', 1),
31 ('Data Structures', 2),
32 ('Databases', 3),
33 ('Web Dev', 1);
34
35 INSERT INTO students(name, email, course_id) VALUES
36 ('David', 'david@gmail.com', 1),
37 ('Emma', 'emma@gmail.com', 2),
38 ('Frank', 'frank@gmail.com', 3);
39
40 -- * BEFORE UPDATE
41 SELECT * FROM students;
42 -- * UPDATE
43 UPDATE students
44 SET email = 'emma_new@gmail.com'
45 WHERE name = 'Emma';
46 -- * AFTER UPDATE
47 SELECT * FROM students;
48
49 -- * BEFORE DELETE
50 SELECT * FROM courses;
51 -- * DELETE
52 DELETE FROM courses
53 WHERE name = 'Web Dev';
54 -- * AFTER DELETE
55 SELECT * FROM courses;
```

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 3 of 3

id	name	email	course_id
1	David	david@gmail.com	1
2	Emma	emma@gmail.com	2
3	Frank	frank@gmail.com	3

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 3 of 3

id	name	email	course_id
1	David	david@gmail.com	1
2	Emma	emma_new@gmail.com	2
3	Frank	frank@gmail.com	3

SQL v < 1 / 1 > 1 - 4 of 4

id	name	instructor_id
1	Programming	1
2	Data Structures	2
3	Databases	3
4	Web Dev	1

SQL v < 1 / 1 > 1 - 0 of 0

SQL v < 1 / 1 > 1 - 3 of 3

id	name	instructor_id
1	Programming	1
2	Data Structures	2
3	Databases	3

Observation:

The CRUD operations were successfully performed on the database. The SELECT query retrieved all course records. The UPDATE operation modified a student's email address correctly, which was verified using a SELECT query. The DELETE operation removed a course from the database, and the change was confirmed by querying the courses table. No unintended data loss occurred during these operations.

Task 4: Reporting Using Aggregate Queries

Prompt:

Generate SQL queries to produce summary reports using aggregate functions. The queries should count the number of students enrolled in each course and list courses that have more than a specified number of enrollments. Use GROUP BY and HAVING clauses appropriately and verify the correctness of the results.

Code:

```
1 -- Clean old tables
2 DROP TABLE IF EXISTS students;
3 DROP TABLE IF EXISTS courses;
4
5 -- create tables
6 CREATE TABLE courses (
7     id INTEGER PRIMARY KEY AUTOINCREMENT,
8     name TEXT
9 );
10
11 CREATE TABLE students (
12     id INTEGER PRIMARY KEY AUTOINCREMENT,
13     name TEXT,
14     course_id INTEGER
15 );
16
17 -- Insert courses
18 INSERT INTO courses(name) VALUES
19 ('Programming'),
20 ('Data Structures'),
21 ('Databases'),
22 ('Web Dev');
23
24 -- Insert students
25 INSERT INTO students(name, course_id) VALUES
26 ('David', 1),
27 ('Emma', 1),
28 ('Frank', 2),
29 ('Grace', 2),
30 ('Helen', 2),
31 ('Ivy', 3);
32
33 -- CHECK DATA FIRST
34 SELECT * FROM courses;
35 SELECT * FROM students;
36
37
38 -- COUNT students per course
39 SELECT courses.name,
40        COUNT(students.id) AS total_students
41 FROM courses
42 LEFT JOIN students ON courses.id = students.course_id
43 GROUP BY courses.name;
44
45
46 -- COURSES with more than 2 students
47 SELECT courses.name,
48        COUNT(students.id) AS total_students
49 FROM courses
50 LEFT JOIN students ON courses.id = students.course_id
51 GROUP BY courses.name
52 HAVING COUNT(students.id) > 2;
```

id	name
1	Programming
2	Data Structures
3	Databases
4	Web Dev

id	name	course_id
1	David	1
2	Emma	1
3	Frank	2
4	Grace	2
5	Helen	2
6	Ivy	3

name	total_students
Data Structures	3
Databases	1
Programming	2
Web Dev	0

name	total_students
Data Structures	3

id	name
1	Programming
2	Data Structures
3	Databases
4	Web Dev

id	name	course_id
1	David	1
2	Emma	1
3	Frank	2
4	Grace	2
5	Helen	2
6	Ivy	3

name	total_students
Data Structures	3
Databases	1
Programming	2
Web Dev	0

name	total_students
Data Structures	3

Observation:

Aggregate queries were successfully used to generate summary reports. The GROUP BY clause was used to group students by course, and the COUNT function calculated the number of students in each

course. The HAVING clause was applied to filter courses with more than a specified number of enrollments. The results were accurate and matched the inserted data.

Task 5: Query Optimization with AI Suggestions

Prompt:

Analyze an existing SQL query that retrieves courses taught by instructors from a specific country. Refactor the query using efficient joins and suggest indexing techniques to improve performance. Compare the results before and after optimization to ensure correctness and improved readability.

Code:

The screenshot shows a SQL IDE with a script editor on the left and a query runner on the right. The script includes table creation, data insertion, and two queries: an original query using a subquery and an optimized query using a JOIN. The query runner shows the results of these queries, which are identical.

```
1 -- Clean old tables
2 DROP TABLE IF EXISTS courses;
3 DROP TABLE IF EXISTS instructors;
4
5 -----
6 -- create tables
7 CREATE TABLE instructors (
8   id INTEGER PRIMARY KEY AUTOINCREMENT,
9   name TEXT,
10  country TEXT
11 );
12
13 CREATE TABLE courses (
14   id INTEGER PRIMARY KEY AUTOINCREMENT,
15   name TEXT,
16   instructor_id INTEGER,
17   FOREIGN KEY (instructor_id) REFERENCES instructors(id)
18 );
19
20 -----
21 -- insert sample data
22 INSERT INTO instructors(name, country) VALUES
23 ('Alice', 'USA'),
24 ('Bob', 'India'),
25 ('Charlie', 'India');
26
27 INSERT INTO courses(name, instructor_id) VALUES
28 ('Programming', 1),
29 ('Data Structures', 2),
30 ('Databases', 3);
31
32 -----
33 -- * ORIGINAL QUERY (subquery)
34 SELECT name
35 FROM courses
36 WHERE instructor_id IN (
37   SELECT id FROM instructors WHERE country = 'India'
38 );
39
40 -----
41 -- * OPTIMIZED QUERY (JOIN)
42 SELECT courses.name
43 FROM courses
44 JOIN instructors
45 ON courses.instructor_id = instructors.id
```

The query runner shows the following results:

name
Data Structures
Databases

The results are identical for both the original and optimized queries.

```
37 | SELECT id FROM instructors WHERE country = 'India'
38 | );
39 |
40 | -----
41 | -- ♦ OPTIMIZED QUERY (JOIN)
42 | SELECT courses.name
43 | FROM courses
44 | JOIN instructors
45 | ON courses.instructor_id = instructors.id
46 | WHERE instructors.country = 'India';
47 |
48 | -----
49 | -- ♦ CREATE INDEXES (optimization)
50 | CREATE INDEX idx_instructor_country ON instructors(country);
51 | CREATE INDEX idx_course_instructor ON courses(instructor_id);
```

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 2 of 2

name
Data Structures
Databases

SQL ▾ < 1 / 1 > 1 - 2 of 2

name
Data Structures
Databases

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 0 of 0

Observation:

The original query used a subquery to retrieve courses based on instructor country, which can be inefficient for large datasets. The optimized query replaced the subquery with a JOIN, improving readability and performance. Indexes were suggested on frequently used columns such as country and instructor_id to further enhance query speed. Both queries produced identical results, confirming correctness after optimization.